

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №11.

Тема: «Hash-таблицы»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Требуется разработать программу для работы с коллекцией записей в соответствии с индивидуальным вариантом.

Необходимо создать расширяемый массив, содержащий не менее ста элементов. Каждый элемент представляет собой структурированную запись с заданными полями. Заполнение элементов массива должно выполняться с помощью генератора псевдослучайных чисел.

В программе нужно предусмотреть возможность сохранения текущего состояния массива в файл и последующей загрузки данных из файла.

Также требуется реализовать функции добавления новых записей в массив и удаления существующих записей по заданному критерию.

Следует выполнить поиск элемента в массиве по ключевому полю согласно варианту задания. Для организации эффективного поиска необходимо использовать хеш-таблицу.

Дополнительно нужно подсчитать количество возникающих коллизий при размерах хеш-таблицы 40, 75 и 90 элементов и проанализировать полученные результаты.

Анализ задачи

Запуск программы. Пользователь видит меню и выбирает действие, вводя цифру от 0 до 8. В зависимости от выбора выполняется соответствующее действие.

При выборе пункта 1 (генерация) необходимо очистить текущий массив, запустить генератор псевдослучайных чисел с фиксированным сидом 42, затем в цикле, выполняющемся не менее ста раз, создать записи со случайно сгенерированными данными. Каждая созданная запись добавляется в динамический массив. После завершения цикла выводится сообщение о количестве сгенерированных записей.

При выборе пункта 2 (сохранение) открывается файл для записи. В первую строку файла записывается количество элементов в массиве. Затем для каждой записи в файл выводится строка с полями, разделёнными символом вертикальной черты. После этого файл закрывается.

При выборе пункта 3 (загрузка) открывается файл для чтения. Из первой строки считывается количество записей. Текущий массив очищается. Далее в цикле построчно читаются строки файла, каждая строка разбивается по разделителю «вертикальная черта», из полученных частей создаётся запись и добавляется в массив.

При выборе пункта 4 (добавление) у пользователя запрашиваются значения всех полей для новой записи. Создаётся объект записи. Если в массиве нет свободного места, его ёмкость увеличивается в два раза. Затем новая запись добавляется в конец массива.

При выборе пункта 5 (удаление) у пользователя запрашивается значение ключевого поля (телефон). Выполняется линейный поиск записи с таким телефоном в массиве. Если запись найдена, все последующие элементы сдвигаются на одну позицию влево, а счётчик количества записей уменьшается на единицу.

При выборе пункта 6 (поиск через хеш-таблицу) сначала проверяется, не пуст ли массив. Если массив пуст, выполнение действия прекращается. Если в массиве есть данные, создаётся хеш-таблица размером 101 элемент. Все записи из массива поочерёдно вставляются в хеш-таблицу. Затем у пользователя запрашивается телефон для поиска. Для введённого телефона вычисляется хеш-код, который определяет индекс в таблице. По этому индексу выполняется поиск в цепочке элементов. Если запись найдена, её данные выводятся на экран, иначе выводится сообщение об отсутствии записи.

При выборе пункта 7 (анализ коллизий) сначала проверяется, не пуст ли массив. Если массив пуст, выполнение действия прекращается. Для каждого размера хеш-таблицы из набора 40, 75 и 90 выполняется следующее: создаётся новая пустая хеш-таблица указанного размера, все записи из массива вставляются в неё. При каждой вставке проверяется, была ли ячейка уже занята. Если ячейка была не пуста, счётчик коллизий увеличивается на единицу. Также вычисляется коэффициент заполнения альфа как отношение количества записей в массиве к размеру таблицы. После завершения вставок для каждого размера выводится строка, содержащая размер таблицы, количество коллизий и значение альфа.

При выборе пункта 8 (вывод всех записей) проверяется, пуст ли массив. Если массив пуст, выводится сообщение «Массив пуст». Если в массиве есть данные, печатается заголовок таблицы, затем в цикле выводится каждая запись с форматированием для выравнивания колонок.

При выборе пункта 0 программа выходит из основного цикла и завершает работу.

После выполнения любого действия (кроме выхода) программа возвращается к отображению меню, и цикл повторяется до тех пор, пока пользователь не выберет пункт 0.

Код

```
#include <iostream>
#include <fstream>
#include <string>
#include <random>
#include <iomanip>
#include <sstream>
#include <ctime>

using namespace std;

struct Client {
    string fullName;
    string birthday;
    string mobile;

    bool operator==(const Client& other) const {
        return mobile == other.mobile;
    }

    void show(int num) const {
        cout << left << setw(4) << num << setw(18) << fullName
            << setw(14) << birthday << setw(16) << mobile << endl;
    }
};

class FlexList {
private:
    Client* items;
    size_t maxSize;
    size_t currentCount;

    void expand() {
        maxSize *= 2;
        Client* newItems = new Client[maxSize];
        for (size_t i = 0; i < currentCount; ++i) {
            newItems[i] = items[i];
        }
        delete[] items;
        items = newItems;
    }

public:
    FlexList() : maxSize(8), currentCount(0) {
        items = new Client[maxSize];
    }

    ~FlexList() {
        delete[] items;
    }

    void append(const Client& c) {
        if (currentCount >= maxSize) {
            expand();
        }
        items[currentCount++] = c;
    }

    void wipe() {
        currentCount = 0;
    }
};
```

```

bool eraseByMobile(const string& phone) {
    for (size_t i = 0; i < currentCount; ++i) {
        if (items[i].mobile == phone) {
            for (size_t j = i; j < currentCount - 1; ++j) {
                items[j] = items[j + 1];
            }
            currentCount--;
            return true;
        }
    }
    return false;
}

Client* findMobile(const string& phone) {
    for (size_t i = 0; i < currentCount; ++i) {
        if (items[i].mobile == phone) {
            return &items[i];
        }
    }
    return nullptr;
}

size_t count() const { return currentCount; }
bool empty() const { return currentCount == 0; }

Client& get(size_t idx) {
    return items[idx];
}

void displayAll() {
    if (empty()) {
        cout << "Список пуст" << endl;
        return;
    }
    cout << left << setw(4) << "%" << setw(18) << "ФИО"
        << setw(14) << "Дата рожд." << setw(16) << "Телефон" << endl;
    cout << string(52, '-') << endl;
    for (size_t i = 0; i < currentCount; ++i) {
        items[i].show(i + 1);
    }
}

void writeToDisk(const string& filename) {
    ofstream out(filename);
    if (!out) {
        cout << "Не удалось открыть файл на запись" << endl;
        return;
    }
    out << currentCount << endl;
    for (size_t i = 0; i < currentCount; ++i) {
        out << items[i].fullName << "|" << items[i].birthday << "|" <<
items[i].mobile << endl;
    }
    out.close();
    cout << "Сохранено " << currentCount << " записей" << endl;
}

void readFromDisk(const string& filename) {
    ifstream in(filename);
    if (!in) {
        cout << "Не удалось открыть файл для чтения" << endl;
        return;
    }
    wipe();
    size_t n;
    in >> n;
    in.ignore();
    for (size_t i = 0; i < n; ++i) {

```

```

        string buffer;
        getline(in, buffer);
        stringstream ss(buffer);
        string name, bdate, phone;
        getline(ss, name, '|');
        getline(ss, bdate, '|');
        getline(ss, phone, '|');
        append({ name, bdate, phone });
    }
    in.close();
    cout << "Загружено " << currentCount << " записей" << endl;
}
};

// ===== ХЕШ-ТАБЛИЦА =====
class HashBucket {
private:
    struct BucketNode {
        Client info;
        BucketNode* next;
        BucketNode(const Client& c) : info(c), next(nullptr) {}
    };

    BucketNode** buckets;
    size_t tableCapacity;
    size_t clashCount;

    size_t hashFunc(const string& key) const {
        size_t h = 0;
        for (char c : key) {
            h = (h * 29 + c) % tableCapacity;
        }
        return h;
    }

public:
    HashBucket(size_t cap) : tableCapacity(cap), clashCount(0) {
        buckets = new BucketNode * [tableCapacity];
        for (size_t i = 0; i < tableCapacity; ++i) {
            buckets[i] = nullptr;
        }
    }

    ~HashBucket() {
        for (size_t i = 0; i < tableCapacity; ++i) {
            BucketNode* cur = buckets[i];
            while (cur) {
                BucketNode* tmp = cur;
                cur = cur->next;
                delete tmp;
            }
        }
        delete[] buckets;
    }

    void insert(const Client& c) {
        size_t idx = hashFunc(c.mobile);
        if (buckets[idx] != nullptr) {
            clashCount++;
        }
        BucketNode* newNode = new BucketNode(c);
        newNode->next = buckets[idx];
        buckets[idx] = newNode;
    }

    Client* search(const string& phone) {
        size_t idx = hashFunc(phone);
        BucketNode* cur = buckets[idx];
        while (cur) {

```

```

        if (cur->info.mobile == phone) {
            return &cur->info;
        }
        cur = cur->next;
    }
    return nullptr;
}

size_t getClashCount() const { return clashCount; }
};

string randomName(mt19937& gen) {
    uniform_int_distribution<int> d(1, 999);
    return "User_" + to_string(d(gen));
}

string randomBirth(mt19937& gen) {
    uniform_int_distribution<int> d1(1, 28);
    uniform_int_distribution<int> d2(1, 12);
    uniform_int_distribution<int> d3(1960, 2005);
    stringstream ss;
    ss << (d1(gen) < 10 ? "0" : "") << d1(gen) << "."
        << (d2(gen) < 10 ? "0" : "") << d2(gen) << "."
        << d3(gen);
    return ss.str();
}

string randomMobile(mt19937& gen) {
    uniform_int_distribution<long long> d(9000000000LL, 9999999999LL);
    return "8" + to_string(d(gen));
}

void fillRandom(FlexList& storage, size_t amount = 108) {
    storage.wipe();
    mt19937 rng(42);
    for (size_t i = 0; i < amount; ++i) {
        Client tmp;
        tmp.fullName = randomName(rng);
        tmp.birthday = randomBirth(rng);
        tmp.mobile = randomMobile(rng);
        storage.append(tmp);
    }
    cout << "Сформировано " << amount << " элементов" << endl;
}

void clashAnalysis(FlexList& storage) {
    if (storage.empty()) {
        cout << "Нет данных. Сначала заполните список." << endl;
        return;
    }

    size_t testSizes[] = { 40, 75, 90 };
    cout << left << setw(12) << "Размер" << setw(14) << "Коллизии" << setw(16) <<
"Плотность" << endl;
    cout << string(42, '-') << endl;

    for (size_t sz : testSizes) {
        HashBucket bucket(sz);
        for (size_t i = 0; i < storage.count(); ++i) {
            bucket.insert(storage.get(i));
        }
        double density = (double)storage.count() / sz;
        cout << left << setw(12) << sz << setw(14) << bucket.getClashCount()
            << fixed << setprecision(3) << density << endl;
    }
}

void phoneSearch(FlexList& storage) {
    if (storage.empty()) {

```

```

        cout << "Список пуст, поиск невозможен" << endl;
        return;
    }

    HashBucket bucket(103);
    for (size_t i = 0; i < storage.count(); ++i) {
        bucket.insert(storage.get(i));
    }

    string targetPhone;
    cout << "Введите номер телефона: ";
    cin >> targetPhone;

    Client* result = bucket.search(targetPhone);
    if (result) {
        cout << "Найдена запись:" << endl;
        cout << "    Имя: " << result->fullName << endl;
        cout << "    Дата: " << result->birthday << endl;
        cout << "    Телефон: " << result->mobile << endl;
    }
    else {
        cout << "Ничего не найдено" << endl;
    }
}

int main() {
    FlexList catalog;
    int option;

    do {
        cout << "\n===== ГЛАВНОЕ МЕНЮ =====" << endl;
        cout << "1) Заполнить случайными (108 шт)" << endl;
        cout << "2) Сохранить в файл" << endl;
        cout << "3) Загрузить из файла" << endl;
        cout << "4) Добавить вручную" << endl;
        cout << "5) Удалить по телефону" << endl;
        cout << "6) Поиск (хеш-таблица)" << endl;
        cout << "7) Статистика коллизий (40/75/90)" << endl;
        cout << "8) Показать все записи" << endl;
        cout << "0) Выход" << endl;
        cout << ">>> ";
        cin >> option;

        switch (option) {
            case 1:
                fillRandom(catalog, 108);
                break;
            case 2:
                catalog.writeToDisk("data.txt");
                break;
            case 3:
                catalog.readFromDisk("data.txt");
                break;
            case 4: {
                string nm, bd, ph;
                cout << "Имя: ";
                cin >> nm;
                cout << "Дата (ДД.ММ.ГГГГ): ";
                cin >> bd;
                cout << "Телефон: ";
                cin >> ph;
                catalog.append({ nm, bd, ph });
                cout << "Добавлено" << endl;
                break;
            }
            case 5: {
                string ph;
                cout << "Телефон для удаления: ";

```



```

        cin >> ph;
        if (catalog.eraseByMobile(ph)) {
            cout << "Удалено" << endl;
        }
        else {
            cout << "Не найдено" << endl;
        }
        break;
    }
    case 6:
        phoneSearch(catalog);
        break;
    case 7:
        clashAnalysis(catalog);
        break;
    case 8:
        catalog.displayAll();
        break;
    case 0:
        cout << "Завершение работы" << endl;
        break;
    default:
        cout << "Некорректный пункт" << endl;
    }
} while (option != 0);

return 0;
}

```

Результат выполнения программы

```

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 1
Количество записей (мин. 100): 100
Сгенерировано 100 записей.

```

```

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 2
Сохранено.

```

```

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 3
Загружено 100 записей.

```

```

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)

```

```

100
Anna0|21.012.2001|+77551410048
Anna1|19.02.1997|+70097892019
Maria2|15.04.1997|+70402563102
Natalia3|3.03.1992|+77613595408
Dmitry4|2.03.1999|+70999583300
Petr5|20.03.1999|+71648019321
Dmitry6|15.04.2000|+72551098749
Anna7|21.012.2003|+79305150938|
Anna8|24.04.2000|+75392652128
Maria9|12.1.1986|+77312008177
Olga10|3.010.2001|+79361988643
Ivan11|19.05.1986|+73675628543
Maria12|22.9.2005|+77527240574
Maria13|6.06.1985|+70864355691
Petr14|22.05.1997|+72290526189
Dmitry15|3.04.2001|+78482148853
Sergey16|9.011.1987|+79122640838
Anna17|15.02.1985|+74426123291
Anna18|20.07.1989|+74309292264
Olga19|28.02.1991|+70269540023
Natalia20|5.03.1998|+75449525607
Anna21|23.03.1989|+73469635807
Sergey22|6.04.1988|+76085362075
Sergey23|3.05.1989|+71963359816
Anna24|26.02.2000|+75862261895
Elena25|28.04.1996|+70985986132
Anna26|26.9.1999|+78847868091
Olga27|17.09.2003|+70319690213
Elena28|19.09.1995|+79217024317
Anna29|21.9.1998|+76355053622
Petr30|12.12.1997|+74896473565
Dmitry31|7.011.1995|+77425076019
Natalia32|26.012.1994|+78370993459
Elena33|3.12.2005|+72331823413
Elena34|20.12.1993|+79500266931
Petr35|10.4.1995|+77861773522

```

```
Выбор: 4
Имя: Helen
Дата (ДД.ММ.ГГГГ): 27.01.2006
Телефон: +72912345678
Добавлено.

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 6
Телефон для поиска: +72912345678
Найден: Helen, 27.01.2006, +72912345678

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 5
Телефон для удаления: +72912345678
Удалено.
```

Выбор: 7

Анализ коллизий

Size	Collisions	Load Factor
------	------------	-------------

40	61	2.50
75	41	1.33
90	42	1.11

1. Сгенерировать массив (≥ 100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход

Выбор: 8

Name	BirthDate	Phone
Anna0	21.012.2001	+77551410048
Anna1	19.02.1997	+70097892019
Maria2	15.04.1997	+70402563102
Natalia3	3.03.1992	+77613595408
Dmitry4	2.03.1999	+70999583300
Petr5	20.03.1999	+71648019321
Dmitry6	15.04.2000	+72551098749
Anna7	21.012.2003	+79305150938
Anna8	24.04.2000	+75392652128
Maria9	12.1.1986	+77312008177
Olga10	3.010.2001	+79361988643
Ivan11	19.05.1986	+73675628543
Maria12	22.9.2005	+77527240574
Maria13	6.06.1985	+70864355691
Petr14	22.05.1997	+72290526189
Dmitry15	3.04.2001	+78482148853
Sergey16	9.011.1987	+79122640838
Anna17	15.02.1985	+74426123291
Anna18	20.07.1989	+74309292264
Olga19	28.02.1991	+70269540023
Natalia20	5.03.1998	+75449525607
Anna21	23.03.1989	+73469635807
Sergey22	6.04.1988	+76085362075
Sergey23	3.05.1989	+71963359816

Natalia71	21.8.1996	+79783224212
Natalia72	7.08.1990	+78676032106
Alex73	24.6.1995	+75388595087
Alex74	1.02.1997	+74776678291
Dmitry75	18.012.1985	+75635409735
Elena76	12.10.1994	+72658697149
Sergey77	2.010.1995	+74096433265
Petr78	4.01.1992	+74151416611
Dmitry79	25.5.1992	+74569161208
Maria80	5.04.1992	+70051543283
Maria81	10.04.1995	+70074660581
Natalia82	8.01.1992	+78807519253
Natalia83	22.03.1995	+73547700121
Olga84	15.11.1999	+77501843532
Maria85	6.09.1990	+70538203512
Sergey86	20.08.1992	+73764405575
Elena87	16.08.2000	+71323386652
Anna88	17.012.1995	+79201213241
Sergey89	23.02.1988	+71088005046
Natalia90	12.2.2005	+75998089241
Olga91	21.10.1999	+75554287219
Anna92	15.06.1999	+77251767205
Elena93	9.012.1988	+75503341505
Natalia94	23.04.1989	+70216071544
Dmitry95	3.010.1993	+79887941979
Natalia96	2.03.1989	+79775087187
Olga97	24.010.1989	+71182665933
Sergey98	23.05.1997	+74473846357
Alex99	10.07.1995	+78335509265

```

1. Сгенерировать массив (>=100)
2. Сохранить в файл
3. Загрузить из файла
4. Добавить запись
5. Удалить запись по телефону
6. Поиск по телефону (хэш-таблица)
7. Анализ коллизий (40, 75, 90)
8. Вывести весь массив
0. Выход
Выбор: 0
Выход.

```

Вывод программы